

Advanced Threat Monitoring System

A threat monitoring solution for busy chemical researchers

1st Matei Patrascu

ICT Advanced Cybersecurity

Fontys University of Applied Sciences
Eindhoven, Netherlands

m.patrascu@student.fontys.nl

2nd George Leonard Batca

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

g.batca@student.fontys.nl

3rd Afonso Jose Carvalho Marques da Costa

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

a.carvalhomarquesdacosta@student.fontys.nl

4th Miguel Almeida Antunes Mesquita

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

m.almeidaantunesmesquita@student.fontys.nl

5th Norbert Knez

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

n.knez@student.fontys.nl

6th Sebastião Lomelino Rodrigues

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

s.lomelinorodrigues@student.fontys.nl

7th Racxshan Nagalingam

ICT Advanced Cyber Security

Fontys University of Applied Sciences
Eindhoven, Netherlands

r.nagalingam@student.fontys.nl

Abstract—This document provides comprehensive details and implementation of our security stack, an Extended Detection and Response platform built on AWS and intended to be migrated on-premises. The system works in a two-VPC hub-and-spoke architecture that separates the control plane from the data plane.

To verify the platform's detection capability, we conducted penetration tests mapping simulated attack behaviors. This exercise resulted in a set of practical gaps which we document and remediate. Our findings demonstrate both strengths and weaknesses that appear when open source security tooling is integrated at scale.

Index Terms—extended detection and response, intrusion detection, eBPF, Kubernetes security, SIEM, network security monitoring, threat intelligence

I. INTRODUCTION

Big Chemistry runs an automated research environment that plans to deploy its IT infrastructure in cloud. Its systems work together to run experiments and to handle research data that can be sensitive, and most of the production workloads run on Kubernetes clusters on Amazon Web Services (AWS). Because the environment is automated and connected, a single weak spot can give an attacker a way into the rest of it, and the research data itself is the thing that has to be protected.

When the project started, the only real security monitoring in this environment was AWS GuardDuty [1]. GuardDuty is useful, but it only looks at high level log sources, so it has some important blind spots. It does not inspect the actual content of the network traffic, which means an attack that

hides inside normal looking traffic can pass unnoticed. It also has no view of what happens inside the running containers, so things like a process being spawned or a privilege being escalated inside a workload are simply not seen. On top of that, the detection logic cannot really be customised, the data stays inside the AWS service, and the cost grows as the environment grows. In short, the account layer was watched, but the network and the runtime layers were not.

This paper describes how we designed and built a threat monitoring system for Big Chemistry that closes those gaps. Instead of adding another managed product, we built a self hosted Security platform with XDR capabilities out of open source tools. The whole environment is written as Infrastructure as Code, it is split into two separate networks for safety, and it combines detection on the network, detection inside the containers, central correlation of all of that data, threat intelligence, and a response layer.

The main contributions of this work are:

- a reference architecture for cloud native threat monitoring in a segmented AWS environment;
- a working implementation of that architecture on AWS and Kubernetes using free tools;
- a validation of the system through a purple team exercise, with every attack mapped to the MITRE ATT&CK framework;
- a comprehensive analysis of the parts that are not yet finished and the blind spots that remain.

The remainder of this paper is organized as follows. Section II gives the background and the related work. Section III

This work was carried out for Big Chemistry Consortium under the supervision of dr. Ousta Firas.

describes the architecture we designed. Section IV explains how it was implemented. Section V presents the tests and the results. Section VI discusses what the results mean and where the limits are, and Section VII concludes and gives the next steps.

II. BACKGROUND AND RELATED WORK

A. XDR, SIEM and SOAR

Before choosing the tools we first had to be clear about the main ideas behind modern threat monitoring, because the terms SIEM, XDR and SOAR are easy to mix up even though they each solve a different part of the problem. A SIEM (Security Information and Event Management) is the classic central system. It collects logs from many sources, normalises them, and matches them against rules to raise alerts. Its strength is that everything ends up in one searchable place, but it is log centric and it mostly detects rather than acts. XDR (Extended Detection and Response) is a newer idea that extends detection across several layers: network, host, container, and cloud, and correlates them so that related events become one incident instead of many separate alerts. SOAR (Security Orchestration, Automation and Response) solves the other half: once an alert exists, it runs playbooks that act automatically across different tools. These are not competing choices, but complementary layers, and the common pattern is to combine a SIEM or XDR core for detection with a SOAR layer for response.

B. Network Detection: NIDS and NSM

The first gap we had to close was the missing deep packet inspection, so we looked at how detection on the network actually works. Two approaches were analyzed. A network intrusion detection system (NIDS) inspects the traffic in real time and raises an alert when it matches a known attack pattern. It looks inside the packets and compares them against large rule sets, which makes it fast and precise for known threats but blind to anything without a signature. Suricata is the main tool in this category [2]. Network security monitoring (NSM) works the other way around. Instead of only alerting on known bad traffic, it records everything as structured logs per protocol, which makes it possible to investigate and to spot behaviour that has no signature, like command and control beaconing or DNS tunneling. Zeek is the tool we chose in this category [3]. These two are a pair rather than competitors: Suricata says whether traffic is known to be malicious, and Zeek says what exactly happened. A practical limit in a cloud environment is that there is no single point where all the traffic can be captured, and that more and more traffic is encrypted, which is why metadata based monitoring with Zeek stays valuable.

C. Host Detection: eBPF Sensors and HIDS

The network stack does not see what happens once an attacker is already running code on a host, so we also needed visibility at the host level. eBPF (extended Berkeley Packet Filter) has become the industry standard for this use case in

on-prem and IaaS environments because it lets small verified programs attach to the Linux kernel and watch every system call with very low overhead, without installing kernel modules [4]. We studied two eBPF based tools. Falco detects suspicious behaviour in containers and on hosts by watching system calls and applying rules, but it only alerts [5]. Tetragon also works in the kernel, but it can enforce as well as observe, so it can stop a process before it runs [6]. We use Falco to see and Tetragon as a pair to get insights into the processes and autonomously block malicious activity.

The eBPF sensors produce raw kernel events, but raw events from many hosts are not the same as useful alerts. Turning them into something an analyst can work with is the job of a HIDS (host intrusion detection system). A HIDS runs on the host itself and does more than read sensor output: it also collects the local logs, performs file integrity monitoring, and checks the configuration of the machine. In our stack this role is filled by the Wazuh agent [8]. On every node the agent gathers what the sensors write and forwards it, and it adds its own host level checks on top. The agents send everything to a central Wazuh manager, which decodes the events, applies detection rules, and stores the resulting alerts. This is also the point where the host based and the network based detection come together, which is exactly the HIDS and NIDS integration that the main research question asks for. The central correlation role of Wazuh is described further in Section III.

D. Network Control: SDN and Cilium

Another part of monitoring is having dynamic control over the network itself. Software Defined Networking (SDN) is one way to do this. SDN separates the control plane of the network from the data plane, so policies can be defined centrally and changed on the fly, which is attractive for security because the network can react to threats. In our case, though, classic SDN does not fit. The environment runs in public cloud, where the provider's network is not directly programmable, and a central SDN controller would be a single point of failure in a Kubernetes cluster where workloads and IP addresses change constantly. Instead we use Cilium, which is built on eBPF and runs its data plane in the kernel of each node [7]. Network rules are written against labels instead of IP addresses, every node enforces them locally so there is no central controller, and the network stays programmable. In this way we keep the idea behind SDN, central and dynamic control, in a form that fits a cloud native environment.

E. Threat Intelligence Sharing

Detection is only as good as the threat data behind it, so the last building block is threat intelligence. Before dedicated platforms existed, indicators were shared informally through emails and reports, which machines cannot easily process. A threat intelligence platform stores this data in a structured, machine readable form and shares it through standard formats and APIs. MISP is the open source platform we chose [9]. It stores indicators of compromise such as malicious IP addresses,

domains and file hashes, and it can export them directly into Suricata rules and Zeek intelligence entries, which is exactly the hook we use to feed the network sensors automatically.

III. SYSTEM ARCHITECTURE

This section describes the architecture we designed and the reasoning behind the main choices. It explains what the system looks like and why, while the details of how it was actually built are left for Section IV. The full reference architecture is shown in Fig. 1.

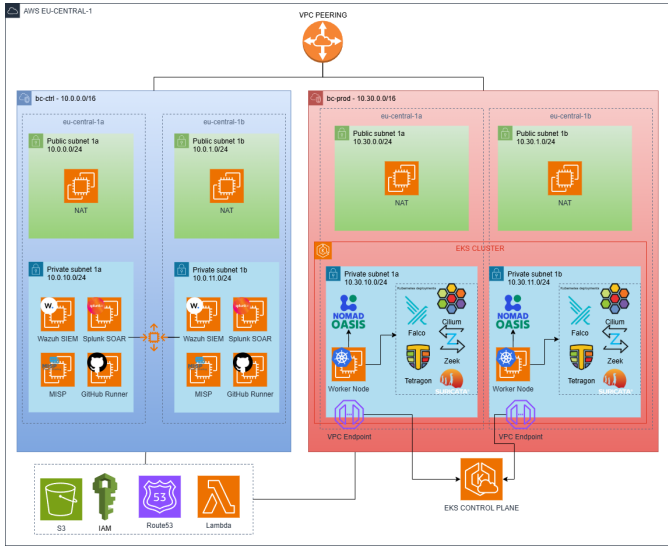


Fig. 1. Reference architecture. A separate control plane holds the monitoring backend, while sensors run on every node of the production plane and send their telemetry to a central SIEM.

A. Segmented Network Design

The first decision we made was to split the environment into two separate networks that are connected by VPC peering. One is the control plane, which holds the monitoring backend: the central SIEM, SOAR, the threat intelligence platform Wazuh and the build server - the "brain" of the system. The other is the production plane, which holds the Kubernetes cluster with the workloads that we actually want to monitor. The reason for this split is security. The systems that store and analyse all the security data are kept apart from the systems that run the workloads and that are exposed to attacks, so if something in the production plane gets compromised, the monitoring backend is not sitting in the same network. The two networks can still reach each other over the private link, and a private DNS zone lets the sensors find the SIEM by name instead of by a fixed address.

B. Distributed Sensors

Inside the production cluster, every node runs a set of sensors as a DaemonSet, which simply means one copy of each sensor runs on every node. This is our answer to the problem that in a cloud environment there is no single point where all the traffic can be captured. Instead of one central tap,

we spread the sensors over the whole cluster. There are four sensor types, and each one covers a different layer. Suricata does the deep packet inspection and raises an alert when it sees a known attack pattern. Zeek records the traffic as structured logs, so we keep the context and can also detect behaviour that has no signature. Falco watches what happens inside the containers at the system call level and alerts on suspicious runtime behaviour. Tetragon also works in the kernel, but it can do more than report, because it can actively stop a process instead of only alerting on it. Together these four cover the network layer and the runtime layer that were blind in the old situation.

C. Network Control and Encryption

For the network itself, the cluster uses Cilium as its network plugin. Cilium is built on eBPF and it lets us write policies that say exactly which workload is allowed to talk to which other workload, using labels instead of IP addresses. This gives the dynamic and programmable control over the network that we wanted, in a way that fits a cloud native environment. On top of that, Cilium encrypts the traffic between the nodes with WireGuard, so the telemetry that travels between the systems is protected.

D. Central Correlation

All the telemetry from the sensors is sent to one central system, Wazuh, which runs in the control plane. On every node a Wazuh agent collects the logs that the sensors write and forwards them to the Wazuh manager. The manager decodes the events, applies the detection rules, and stores everything in its indexer, where it can be searched and shown on a dashboard. This is the one central place where the data from the host based and the network based sensors comes together, which is the point where the HIDS and the NIDS detection are correlated. To keep the cloud visibility that the old situation already had, the design also feeds the AWS account logs, such as CloudTrail and the GuardDuty findings, into the same Wazuh instance, so the deep network and runtime events sit next to the high level account events in one place.

E. Threat Intelligence

The threat intelligence comes from MISP, which also runs in the control plane. In the design, the network sensors pull the current indicators of compromise from MISP on a regular schedule and turn them into Suricata rules and Zeek intelligence entries. This way the community threat intelligence automatically becomes local detection, and nobody has to update the rules by hand.

F. Response Layer

For the response part, the design works on two levels. On the runtime layer, Tetragon enforces directly: with a kernel level policy, a defined set of attacker tools is blocked the moment they try to run, which gives an automatic response on the node itself. On top of that there is an orchestration layer. A SOAR platform receives the high severity alerts from Wazuh

and runs playbooks that can invoke cloud response functions, for example isolating a compromised instance or stripping attacker permissions. Together, the runtime enforcement and the orchestrated response functions are the two halves of the response design.

IV. IMPLEMENTATION

This section describes how the architecture of Section III was actually built. The whole environment is written as code and deployed through CI/CD pipelines, from the two networks down to the security stack itself. Besides of the monitoring components, the production plane also hosts the application used by Big Chemistry, NOMAD Oasis.

A. Infrastructure as Code

The entire platform is written in Terraform and split into two independent configurations, one per network: a control plane and a production plane. Each configuration declares its own networking, compute and security groups, and the remote state for both is stored in a versioned S3 bucket.

The two networks are connected by VPC peering, configured from the production plane side, so that the sensors on the production plane can reach the monitoring backend on the control plane; a private DNS zone shared by both networks lets them find it by name rather than by IP address. The hub & spoke architecture allows for a single point of entry in the network - the prod vpc, as a best effort to design an enterprise-grade network, but that was out of scope.

B. Production Cluster and Network Plane

The workloads and all in-cluster sensors run on a managed Kubernetes cluster. Networking is handled by Cilium. Cilium takes over IP management completely. It also replaces the standard service proxy with its own eBPF implementation, and traffic between nodes is encrypted with WireGuard.

Bootstrapping has a strict order that we build into the pipeline: Cilium goes in before the nodes join, and the security charts and their custom resources are in place before any manifest that depends on them. To enforce this, the pipeline deploys in stages and waits for each one to be ready before moving on.

C. Distributed Sensors

The sensors run as Kubernetes DaemonSets, so every node is covered automatically, including nodes added later. Suricata provides signature-based network detection, Zeek records protocol-level network logs, and Falco and Tetragon watch what happens inside the containers through eBPF. The network sensors write their logs to a shared path on each node, so that everything can be collected.

D. Control-Plane Backend

The monitoring backend runs on virtual machines on the control plane rather than inside the cluster, which keeps it healthy even if the production is down. One host runs the SIEM core (manager, indexer and dashboard together), one runs the threat intelligence platform with its database, and

one runs the response and automation platform. Each host installs and configures itself on first boot from scripts kept in object storage, which makes the machines disposable and reproducible straight from code. None of the credentials live in the images; each host pulls what it needs from the managed secrets store as it sets up.

E. Telemetry Pipeline

The telemetry flows as shown in Fig. 2. Each sensor writes its events to a log file on the node. On every node the Wazuh agent, also deployed as a DaemonSet, reads those log files and forwards the events over the peering link to the Wazuh manager. The manager decodes the events, applies the detection rules, and stores them in the indexer, from where they can be searched and shown on the dashboard.

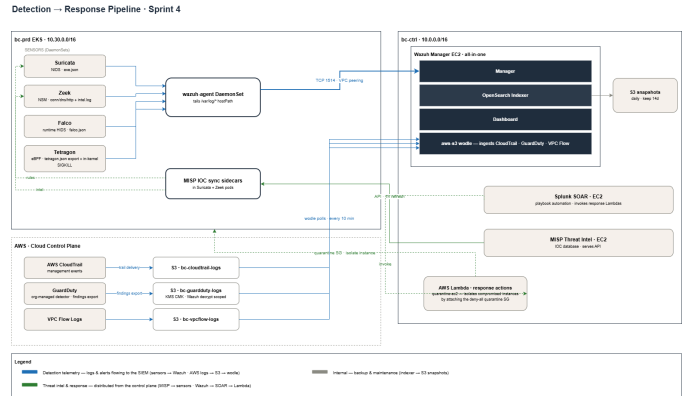


Fig. 2. Telemetry pipeline. Each sensor writes its events to a log on the node; the Wazuh agent forwards them to the manager, which decodes and stores the alerts in the indexer for the dashboard, and forwards the high severity ones to the SOAR.

F. Automated Threat-Intelligence Sync

The network sensors do not get their threat intelligence by hand; it is pulled from MISP automatically. The first problem to solve is access: the sensors need the MISP API key, but we did not want to store that key inside the cluster. A secrets operator running in the cluster solves this. It uses the cluster's own workload identity to read the key from the managed secrets store and make it available to the Suricata and Zeek namespaces, so the key never lives in a manifest or an image. With the key in place, a small sidecar container next to each sensor queries MISP on a fixed schedule, turns the indicators of compromise it gets back into Suricata rules and Zeek intelligence entries, and writes them where the sensor will load them on its next reload. In this way the community threat intelligence becomes local detection on its own, which is exactly the behaviour the design asked for.

G. Detection as Code

Just like the infrastructure, the detection logic lives in the repository and is applied by the pipeline, not typed into the running system by hand. This covers the custom Wazuh rules for the cloud-audit, Kubernetes-audit and runtime events, the

portable detection rules, and the enforcement policies for Tetragon. Two of those Tetragon policies are worth pointing out, because they are the ones doing the active work: one kills a defined set of attacker tools the moment they try to run, and the other watches for privilege escalation such as a process gaining root through `setuid` or changing its capabilities. Keeping the detection in code has a practical benefit: a change to a rule goes through the same review and the same pipeline as any change to the infrastructure, so the detection is versioned and auditable instead of being a manual edit on a live machine.

H. Continuous Deployment

Everything described above is deployed by one pipeline that runs in two stages, and the split exists for a reason. The first stage builds the control plane: the networks, the NAT and DNS, and the backend hosts. It does not just create them and move on, it waits until those hosts are actually reachable before it hands over, because the production plane depends on them. The second stage runs on a self-hosted runner inside the environment and builds the production plane. It creates the cluster, installs the security charts in the strict order described earlier, waits for the custom resources and the secrets webhook to be ready, and only then applies the sensor manifests. Building it this way is what lets a deployment start from a completely empty account and finish on its own, without anyone running a manual step in between.

V. EVALUATION

To find out if the system really works, we tested it as a purple team exercise. One of us played the attacker and another watched the dashboards and wrote down, for every attack, whether it was detected, blocked, or missed. We mapped all the attacks to the MITRE ATT&CK framework, so we could measure the coverage in a structured way.

A. The Pipeline Works End to End

The first thing we saw is that the whole pipeline really works from start to finish. During the test the system handled more than 215,000 alerts in 24 hours, all mapped to MITRE ATT&CK, and all four sensor agents were reporting into Wazuh. The runtime sensor alone produced more than 45,000 events. So the main goal, getting the data from the sensors on the nodes all the way to the central dashboard, was reached.

B. Detection Results

Table I shows the most important attacks we ran and the result for each one. The pattern is clear. The runtime layer, Tetragon and Falco, caught the activity inside the containers very well, and the enforcement policy killed the attacker tools the moment they started. The network detection, Suricata, caught the loud attacks that have a clear traffic pattern, like scanning, brute force, and the denial of service floods. The escape and cloud protections also held, because the container could not break out and a pod could not steal the cloud credentials.

C. Blind Spots

The test also showed honestly where the system does not yet reach. There were two blind spots that stayed open at the end. The first is encrypted command and control over normal web traffic on port 443. Because the traffic is encrypted before the network sensors can read it, this kind of hidden channel is missed. The second is the cloud control plane. The AWS account logs are supposed to be read into Wazuh, but during the test we found that this part was not actually running, so attacks against the cloud account did not raise an alert.

D. False Positives

We also looked at how much noise the system makes, because a monitoring system that floods the analyst with useless alerts is not usable in practice. Most of the alerts were relevant, but one rule, the privilege escalation rule on `setuid`, fired far too often and buried the more important alerts. This is a real false positive problem and it is on our list of things to improve. We did not measure a false positive rate over a longer period, so that is left as future work that needs to be done in collaboration with the scientists.

E. Compared to the Original Situation

The last step was to compare the result with the problem we started from. The deep packet inspection and the runtime visibility that the original setup did not have are now there, and we proved that they fire on real attacks. The data is also centralised and self hosted. But the test also showed that not everything is covered: the encrypted network traffic and the cloud audit trail are still blind spots, and the automated response works on the runtime layer while the orchestrated response across the whole system is not yet switched on.

VI. DISCUSSION AND LIMITATIONS

A. What the System Delivers

The most important value of the result is the extra detection depth compared to the starting point. The original setup only looked at high level logs, with no deep packet inspection and no view inside the containers. The evaluation showed that both of these gaps are now really covered: the network sensors catch scanning, brute force and denial of service, and the runtime sensors catch and even block what happens inside the containers. On top of that, all of this data comes together in one central place, where events from different layers can be seen next to each other.

A second value is the cost. The whole platform is self hosted and built from open source tools, so there are no licensing fees per agent or per gigabyte. The running cost is mainly the cost of the cloud infrastructure, which is predictable and stays around a few hundred dollars per month. Commercial XDR and SIEM products usually charge based on the amount of data that is ingested, which can grow into very high yearly costs as the environment grows. A third value is portability: because the stack is open source and not tied to one cloud provider, it can in principle be moved to a bare metal or on

TABLE I
EXECUTED ATTACK SCENARIOS AND DETECTION RESULTS

Attack (technique)	MITRE	Sensor	Result
Run nmap or nc inside a container	T1059	Tetragon	Blocked
Spawn a shell inside a container	T1059.004	Tetragon / Falco	Detected
Read service account token and exfiltrate it	T1552.001	Tetragon	Detected
Privilege escalation (setuid)	T1548	Tetragon	Detected
Container escape attempt	T1611	runc (patched)	Blocked
Pod tries to steal cloud credentials	T1552.005	Cilium	Blocked
Persistence with a Kubernetes CronJob	T1053.007	Wazuh k8s-audit	Detected
Port scan from a pod	T1046	Suricata	Detected
HTTP login brute force	T1110	Suricata	Detected
Denial of service floods (SYN/UDP/ICMP)	T1498	Suricata	Detected
Command and control on an odd port	T1071	Cilium	Blocked and alarmed
DNS tunneling out of a pod	T1048.003	Zeek / CoreDNS	Detected
Lateral movement between app services	T1210	Hubble	Visible, not blocked
Encrypted command and control over 443	T1071.001	Suricata / Zeek	Missed
Cloud control plane events (CloudTrail)	T1562	Wazuh aws-s3	Missed

premise environment later, which fits the future plans of the environment even though that part was out of scope here.

B. Limitations

We also want to be honest about the weak points. The first limitation is that the central SIEM runs on a single machine, which makes it a single point of failure. This is not only theoretical: during the test the sensor agents lost their connection to the SIEM for a while, and the dashboard went blind until we fixed it. A production setup would need a high availability design.

The second limitation is in the cloud layer. The detection rules for the AWS account are written, but during the test the account logs were not actually being ingested, so attacks against the cloud account did not raise any alert.

The third limitation is the transport security between the components. The channel between the agents and the SIEM is not encrypted by itself, and the threat intelligence sync skips the certificate check. The traffic between the nodes is protected by WireGuard, so it is not fully open, but these points are still real weaknesses.

The fourth limitation is the automated response. The runtime enforcement works well, because Tetragon kills a defined set of attacker tools in the kernel the moment they start, and the response functions were built and tested on their own. What is missing is the last step, activating the playbooks so that an alert triggers the response automatically, so in practice the cross system response still had to be started by hand. Finally, there are a few more specific limits: the network sensors are blind to encrypted content, the runtime enforcement is based on the name of the program so an attacker could rename or copy a tool to get around it, and the validation was done in one lab over a short period, so there is no long term measurement of the false positive rate yet.

VII. CONCLUSION AND FUTURE WORK

We set out to build a more advanced monitoring system for a cloud based research environment whose detection relied only

on high level cloud logs. We designed a reference architecture and built a working version on AWS that combines host and network detection, central correlation, threat intelligence, and runtime enforcement, for a predictable monthly cost. The dynamic network control is provided by the eBPF based plugin Cilium instead of a classic SDN controller, which fits the cloud native environment better. The system was validated with a purple team exercise mapped to MITRE ATT&CK, and it clearly improves on the original situation. Not everything was fully reached: the automated response is only partly there, the cloud account logs were not being ingested during the test, and the on premise part was out of scope.

From the validation we also got a clear list of next steps. The most direct one is finishing the automatic response, by turning on the SOAR playbooks and the Wazuh active response so that an alert triggers a response on its own, since the pieces are already in place. The cloud monitoring should be repaired by fixing the log ingestion, so the rules that are already written finally get data. The encrypted traffic blind spot is best handled with prevention, by locking down the outbound traffic of the containers with an allow list and adding DNS logging, and the runtime enforcement should be made stronger so it cannot be bypassed by simply renaming a tool. The central monitoring should be made highly available so the detection does not stop if one machine goes down, and the alerts should be archived to long term storage. Beyond fixing the open points, two directions are interesting for future work: smarter correlation that combines events from different sensors into one alert, and extending the system to the future on premise environment. Together these steps would turn the platform from a strong detection system into a complete detection and response solution.

REFERENCES

- [1] Amazon Web Services, "Amazon GuardDuty Documentation." [Online]. Available: <https://docs.aws.amazon.com/guardduty/>
- [2] Open Information Security Foundation, "Suricata User Guide." [Online]. Available: <https://docs.suricata.io/>

- [3] The Zeek Project, “Zeek Documentation.” [Online]. Available: <https://docs.zeek.org/>
- [4] eBPF Community, “What is eBPF?,” 2024. [Online]. Available: <https://ebpf.io/what-is-ebpf/>
- [5] The Falco Project, “Falco Documentation,” Cloud Native Computing Foundation. [Online]. Available: <https://falco.org/docs/>
- [6] Cilium Authors, “Tetragon Documentation.” [Online]. Available: <https://tetragon.io/docs/>
- [7] Cilium Authors, “Cilium Documentation,” 2024. [Online]. Available: <https://docs.cilium.io/en/stable/>
- [8] Wazuh, “Wazuh Architecture.” [Online]. Available: <https://documentation.wazuh.com/current/getting-started/architecture.html>
- [9] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, “MISP: The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform,” in *Proc. 2016 ACM Workshop on Information Sharing and Collaborative Security (WISCS)*, 2016, pp. 49–56, doi: 10.1145/2994539.2994542.
- [10] MITRE Corporation, “MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge.” [Online]. Available: <https://attack.mitre.org>